

Operating System

Basic Understanding of the Operating System

By: Dr. Umer Rashid

1

Contents

- Summary
- Introduction to Operating System; Computer System Organization and Architecture
- Operating System Structures and Operations; Process, Memory and Storage management
- Operating System Services; Interface; Calls Types; and Programs
- Operating System Design and implementation; Virtual Machines; and Operating System Generation.

2

Introduction to Operating System

- Operating System
 - A program that acts as
 - an intermediary between a user of a computer and
 - the computer hardware
- Operating System
 - is a software,
 - consisting of programs and data
 - runs on computers
 - manages computer hardware resources
 - provides common services to execute application s/w
- Operating System
 - is the program that,
 - loaded into the computer by a boot program
 - manage all the other programs in a computer

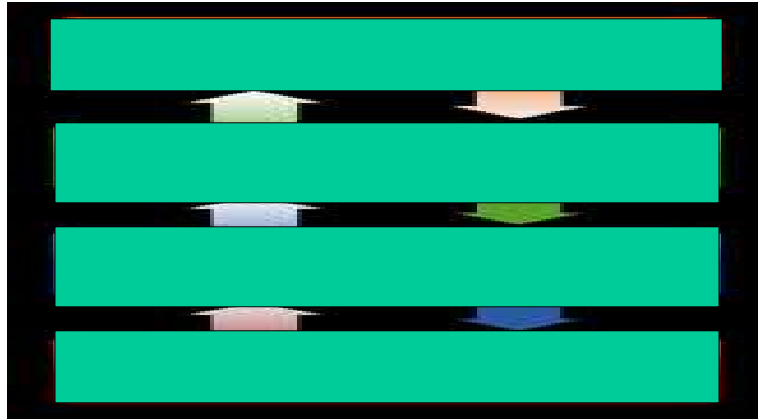
3

Introduction to Operating System

- Computer system can be divided into four components
 - Hardware
 - provides basic computing resources
 - CPU, memory, I/O devices
 - Operating system
 - Controls and coordinates use of hardware among various applications and users
 - Application programs
 - define the ways in which the system resources are used to solve the computing problems of the users
 - Word processors, compilers, web browsers, database systems, video games
 - Users
 - People, machines, other computers

4

Introduction to Operating System



5

Introduction to Operating System

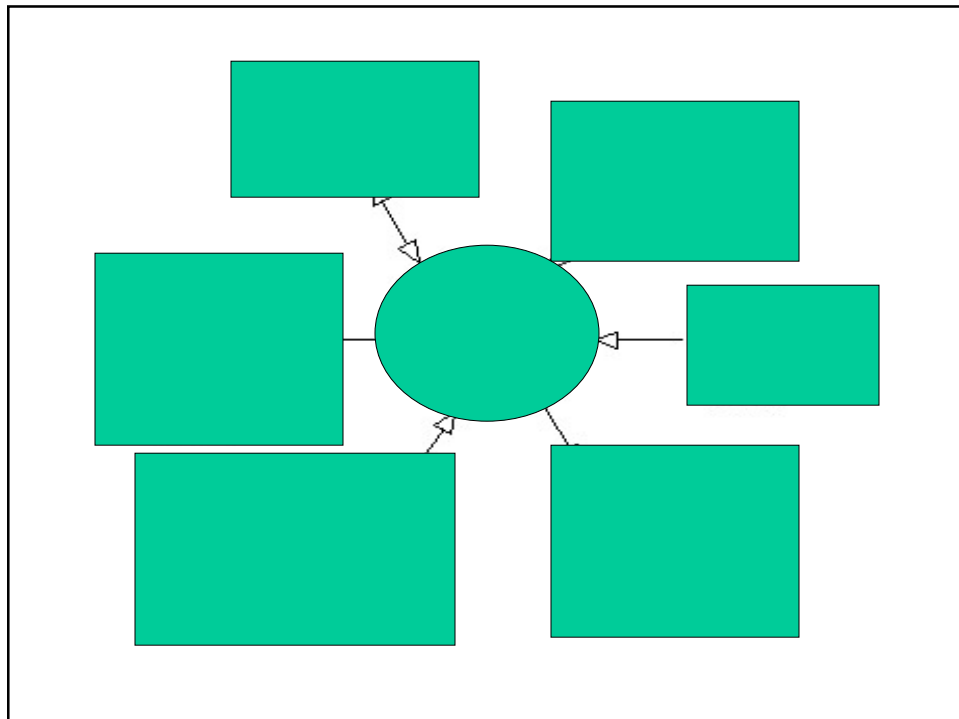
1. Operating System Goals
 - Execute user programs to make solving user problems easier
 - Easier→ ?
 - Make the computer system convenient to use
 - Convenient→ ?
2. Operating System Goals
 - convenience for the user
 - efficient operation of the computer system
3. Operating System Goal
 - optimize performance
 - by making efficient use of the CPU time and memory space
4. Operating System Goals
 - efficient use of a computer system
 - user convenience

6

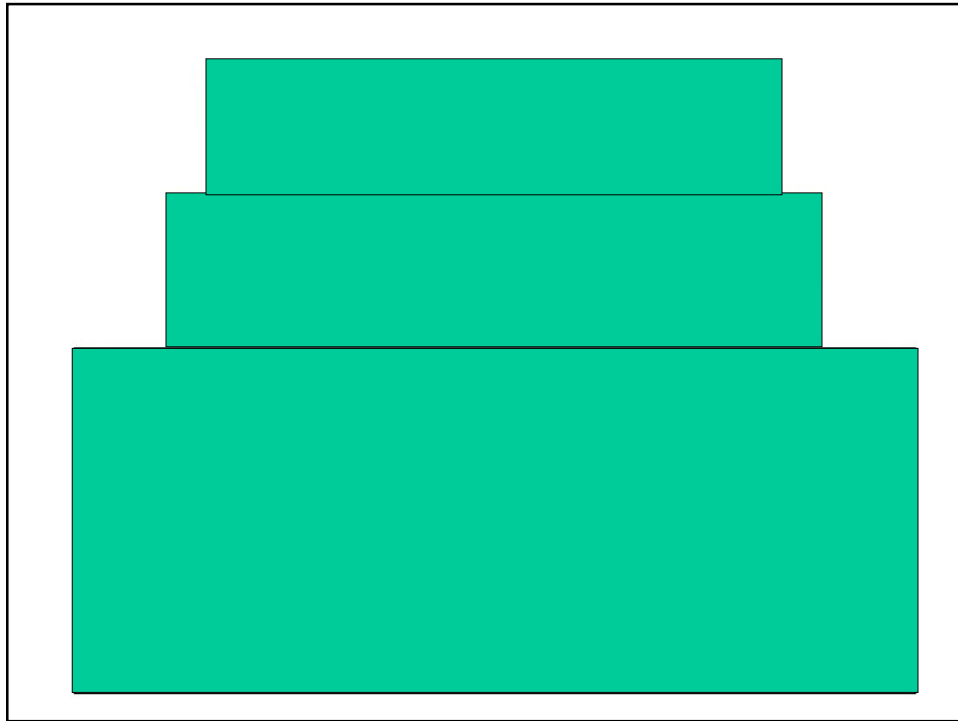
Introduction to Operating System

- Operating System Definition
 - OS is a resource allocator
 - Manages all resources
 - Decides between conflicting requests for efficient and fair resource use
 - OS is a control program
 - Controls execution of programs to prevent errors and improper use of the computer
 - No universally accepted definition
 - “Everything a vendor ships when you order an operating system” is good approximation
 - But varies wildly
 - Program running at all times on the computer” is the kernel.
 - Everything else is either a system program (ships with the operating system) or an application program

7



8

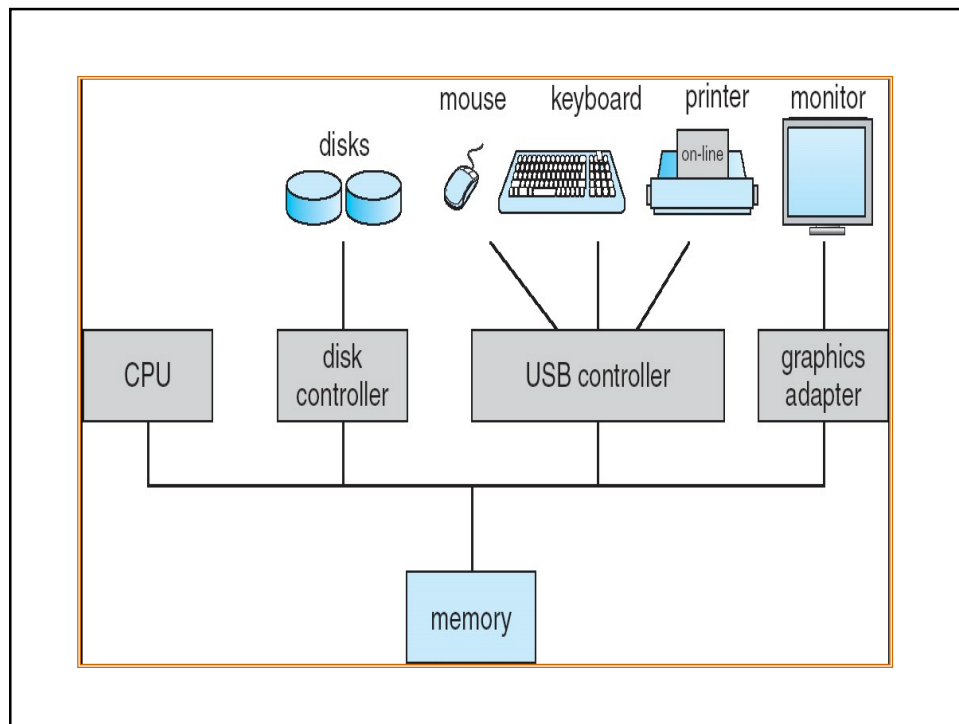


9

Computer System Organization and Architecture

- In a Computer-system
 - One or more CPUs
 - device controllers
 - connect through common busprovides access to shared memory
 - Concurrent execution of CPUs and devices competing for memory cycles

10

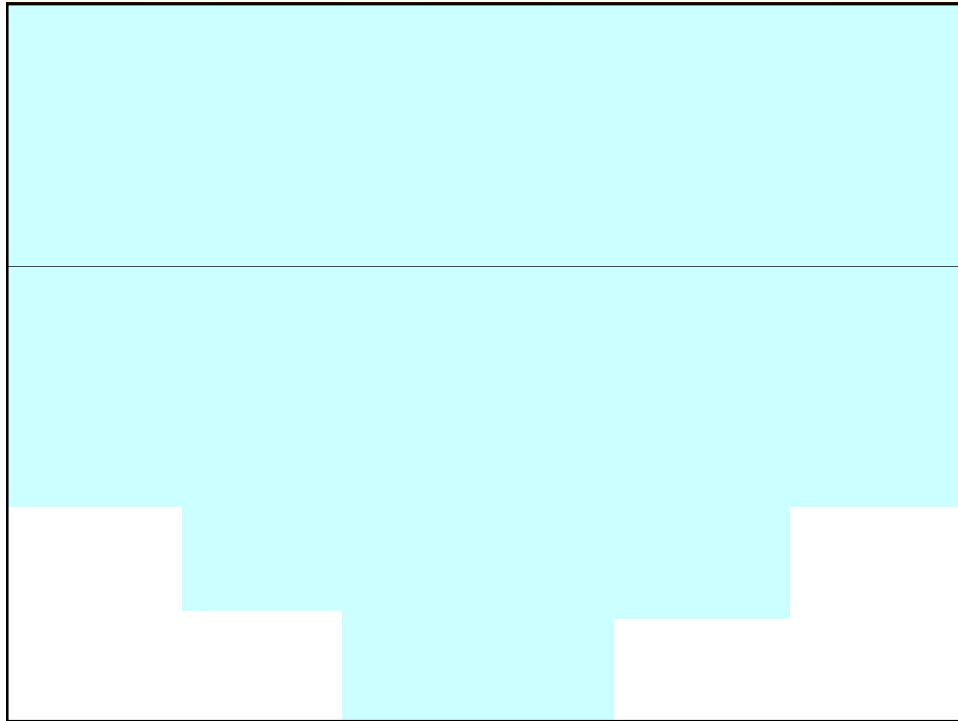


11

Computer System Structure

- Computer system → four components
 - Hardware – provides basic computing resources
 - CPU, memory, I/O devices
 - Operating system
 - Controls and coordinates use of hardware among various applications and users
 - Application programs
 - define the ways in which the system resources are used to solve the computing problems of the users
 - E.g.: Word processors, compilers, web browsers, database systems, video games
 - Users
 - People, machines, other computers

12



13

Computer-System Operation

1. I/O devices and the CPU can execute concurrently
2. Each device controller is in-charge of a particular device type
3. Each device controller has a local buffer.
4. CPU moves data
 1. from/to main memory
 2. to/from local buffers
5. I/O is from the device to local buffer of controller.
6. Device controller informs CPU that it has finished its operation by causing an interrupt

14

Computer-System Operation

- Common Functions of Interrupts
 - Interrupt
 - transfers control to the interrupt service routine generally
 - through the interrupt vector, which
 - contains the addresses of all the service routines
 - Interrupt
 - architecture must save the address of the interrupted or effected instruction
 - Incoming interrupts
 - disabled while another interrupt is being processed to prevent a lost interrupt
 - A trap
 - a software-generated interrupt caused either by an error or a user request
 - An operating system
 - is interrupt driven

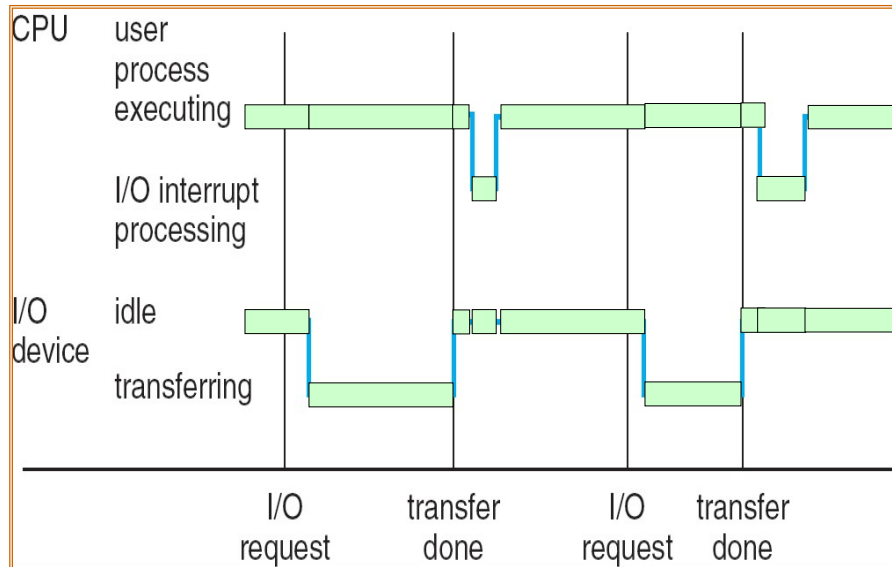
15

Computer-System Operation

- Interrupt Handling
 - The operating system preserves
 - the state of the CPU by storing
 - registers and the program counter.
 - Determines
 - which type of interrupt has occurred:
 - » polling
 - » vectored interrupt system
- Separate segments
 - code determine what action should be taken for each type of interrupt

16

Computer-System Operation



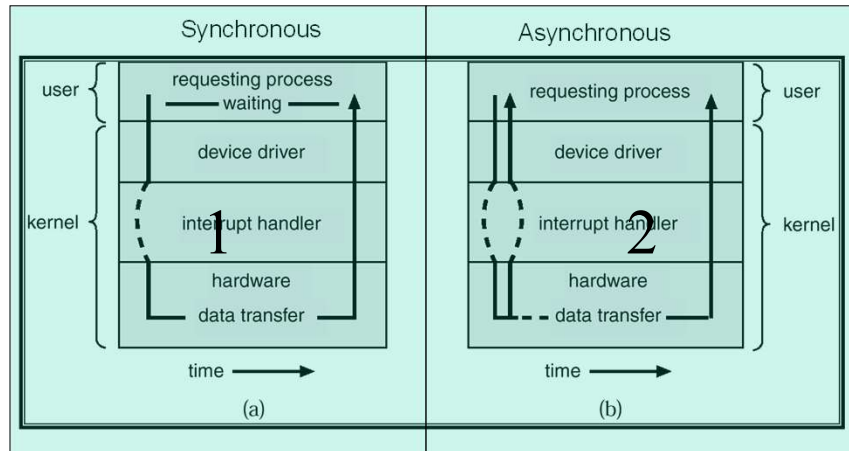
17

Computer System Structure

- I/O Structure → 2 ways
 1. After I/O starts, control returns to user program only upon I/O completion
 - Wait instruction idles the CPU until the next interrupt
 - Wait loop (contention for memory access).
 - At most one I/O request is outstanding at a time, no simultaneous I/O processing
 2. After I/O starts, control returns to user program without waiting for I/O completion
 - System call – request to the operating system to allow user to wait for I/O completion.
 - Device-status table contains entry for each I/O device indicating its type, address, and state.
 - Operating system indexes into I/O device table to determine device status and to modify table entry to include interrupt.

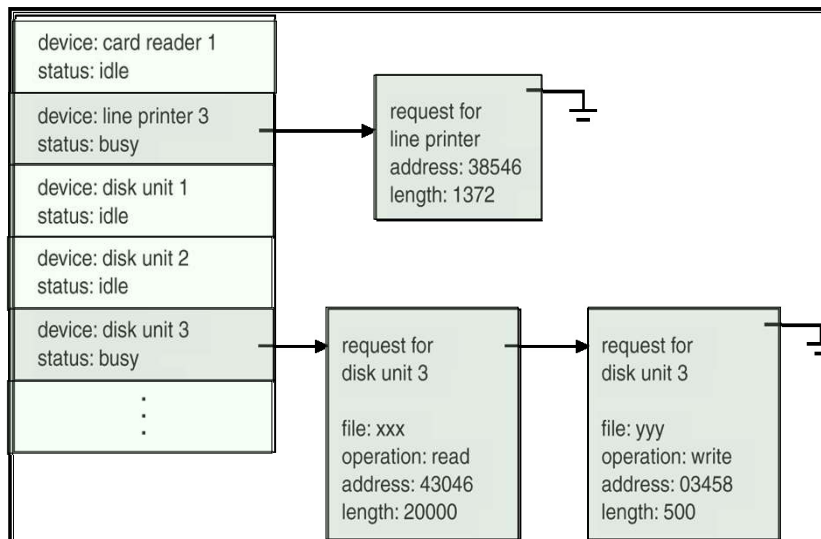
18

Two I/O Methods



19

Device-Status Table



20

Computer System Structure

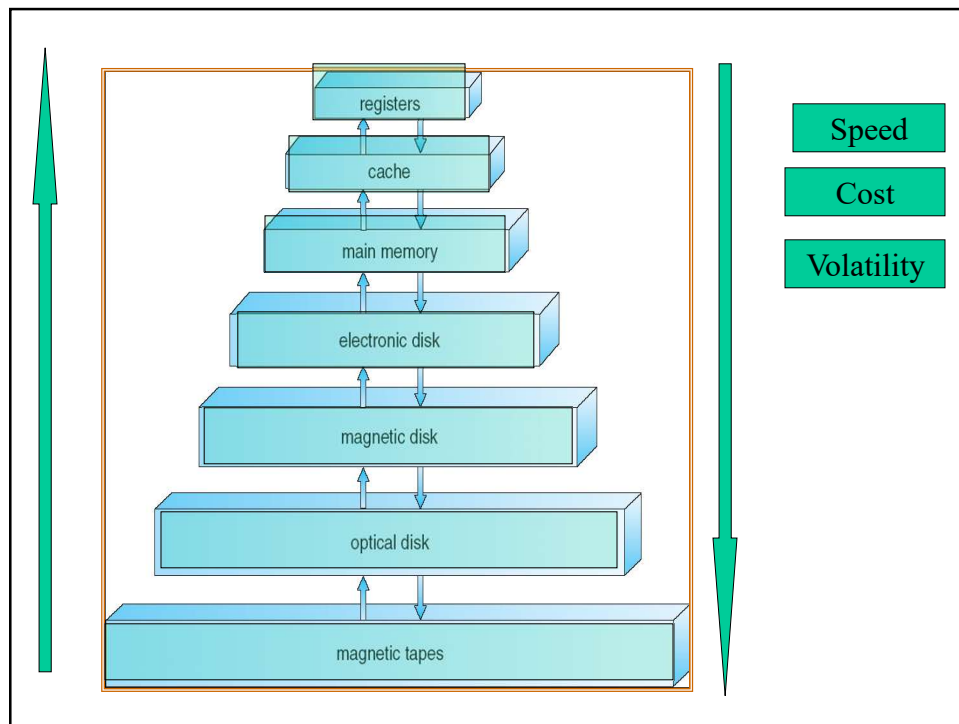
- Direct Memory Access Structure
 - Used for high-speed I/O devices
 - able to transmit information at close to memory speeds
 - Device controller
 - transfers blocks of data from buffer storage directly to main memory without CPU intervention
 - Only on interrupt
 - is generated per block, rather than the one interrupt per byte

21

Computer System Structure

- Storage Structure
 - Main memory → only large storage media that the CPU can access directly
 - Secondary storage → extension of main memory that provides large nonvolatile storage capacity
 - Magnetic disks → rigid metal or glass platters covered with magnetic recording material
 - Disk surface is logically divided into tracks, which are subdivided into sectors
 - The disk controller determines the logical interaction between the device and the computer
 - Caching → copying information into faster storage system; main memory can be viewed as a last cache for secondary storage

22



23

Computer System Structure

- Caching
 - Important principle, performed at many levels in a computer (in hardware, operating system, software)
 - Information in use copied from slower to faster storage temporarily
 - Faster storage (cache) checked first to determine if information is there
 - If it is, information used directly from the cache (fast)
 - If not, data copied to cache and used there
 - Cache smaller than storage being cached
 - Cache management important design problem
 - Cache size and replacement policy

24

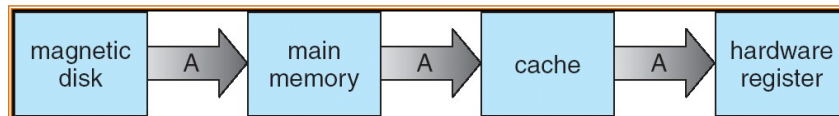
Performance of Various Levels of Storage

Level	1	2	3	4
Name	registers	cache	main memory	disk storage
Typical size	< 1 KB	> 16 MB	> 16 GB	> 100 GB
Implementation technology	custom memory with multiple ports, CMOS	on-chip or off-chip CMOS SRAM	CMOS DRAM	magnetic disk
Access time (ns)	0.25 – 0.5	0.5 – 25	80 – 250	5,000.000
Bandwidth (MB/sec)	20,000 – 100,000	5000 – 10,000	1000 – 5000	20 – 150
Managed by	compiler	hardware	operating system	operating system
Backed by	cache	main memory	disk	CD or tape

25

Issue: Migration of Integer A from Disk to Register

- Multitasking environments
 - must be careful to use most recent value, not matter where it is stored in the storage hierarchy cache



- Multiprocessor environment
 - must provide cache coherency in hardware such that all CPUs have the most recent value
- Distributed environment
 - situation even more complex
 - Several copies of a datum can exist
 - Various solutions covered in Chapter 17

26

Operating System Structures and Operations

- Computer Startup → bootstrap program
 - loaded at power-up or reboot
 - Typically stored in
 - ROM or EEPROM → known as firmware
 - Initializes → all aspects of system
 - Loads → operating system kernel and starts execution
- Executions
 - Multi-tasking
 - Multi-processing
 - Multi-threading
 - Multi-programming
 - Multi-user

27

Operating System Structures and Operations

- Operating System can be
 - **Batch processing**
 - Execution
 - a series of programs or jobs on a computer without manual intervention
 - Support
 1. UNIX
 2. DOS
 3. IBM OS
 - Usage
 1. Data processing → necessary operations
 2. Printing → Bulk documents
 3. Databases → Bulk storage
 4. Images → sequence of operations
 5. Conversion → files from one format to another

28

Operating System can be

- **Single User**
 - Execution
 - Single user processes at any instant of time
 - Support
 1. DOS
 2. Windows
 - Usage
 1. home computer
 2. Work environments
 3. Laptops
- **Uni-programming**
 - Execution
 - One program at a time
 - Support
 1. DOS
 2. Can be → Purpose specific digital systems
 - Usage
 1. Assembly
 2. Embedded systems

29

Operating System can be

- **Uni-processing**
 - Execution
 1. With a single execution central
 - Support
 1. DOS
 2. Windows
 3. Unix
 - Usage
 1. home computer
 2. Work environments
 3. Laptops
- **Multi-tasking**
 - Execution
 - simultaneous performance of two or more tasks by a computer's central
 - Support
 1. Windows 98 and later version
 2. Mac OS
 3. NetWare 6.5
 - Usage
 1. Home Computers
 2. Work Environment
 3. Laptops
 4. Networking

30

Operating System can be

- **Multi-programming**
 - Execution
 1. allocation of a computer system and its resources to more than one concurrent application, job or user
 - Support
 1. Windows
 2. Linux Red Hat
 - Usage
 1. Home
 2. Offices
 3. Networks, etc
- **Multi-user**
 - Execution
 - allows concurrent access by multiple users of a computer
 - Support
 1. Unix
 2. X Window
 3. Concurrent DOS
 4. Multi-user DOS
 5. FlexOS
 - Usage
 - Network based systems with thin clients and fat server

31

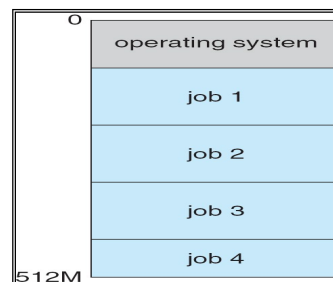
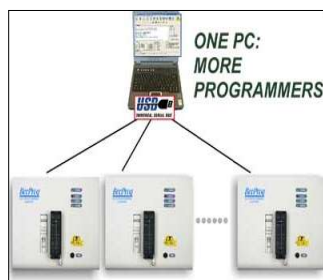
Operating System can be

- **Time-sharing**
 - Execution
 - An operating system that provides sharing of resources among concurrent users
 - Support
 1. IBM systems
 2. Windows
 3. Michigan Terminal System (MTS)
 - Usage
 1. Network based systems with thin clients and fat server
- **Multi-Threading**
 - Execution
 - The ability of an operating system to execute different parts of a program, called threads, simultaneously
 - Support
 - Modern operating systems
 - Usage
 1. Home
 2. Office
 3. Business
 4. Network

32

Operating System Structures and Operations

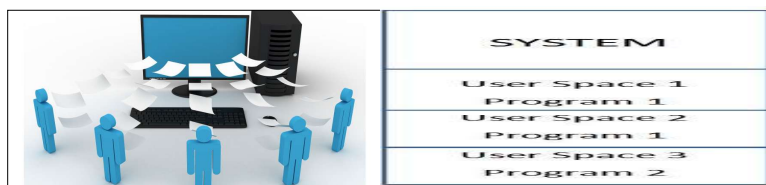
- **Multiprogramming** → needed for efficiency → ?
 - Single user cannot keep CPU and I/O devices busy at all times
 - Multiprogramming organizes jobs (code and data) so CPU always has one to execute
 - A subset of total jobs in system is kept in memory
 - One job selected and run via **job scheduling**
 - When it has to wait (for I/O for example), OS switches to another job



33

Operating System Structures and Operations

- **Timesharing (multitasking)**
 - logical extension in which CPU switches jobs so frequently that users can interact with each job while it is running, creating **interactive** computing
 - **Response time** should be < 1 second
 - Each user has at least one program executing in memory → **process**
 - If several jobs ready to run at the same time → **CPU scheduling**
 - If processes don't fit in memory, **swapping** moves them in and out to run
 - **Virtual memory** allows execution of processes not completely in memory



34

Operating System Structures and Operations

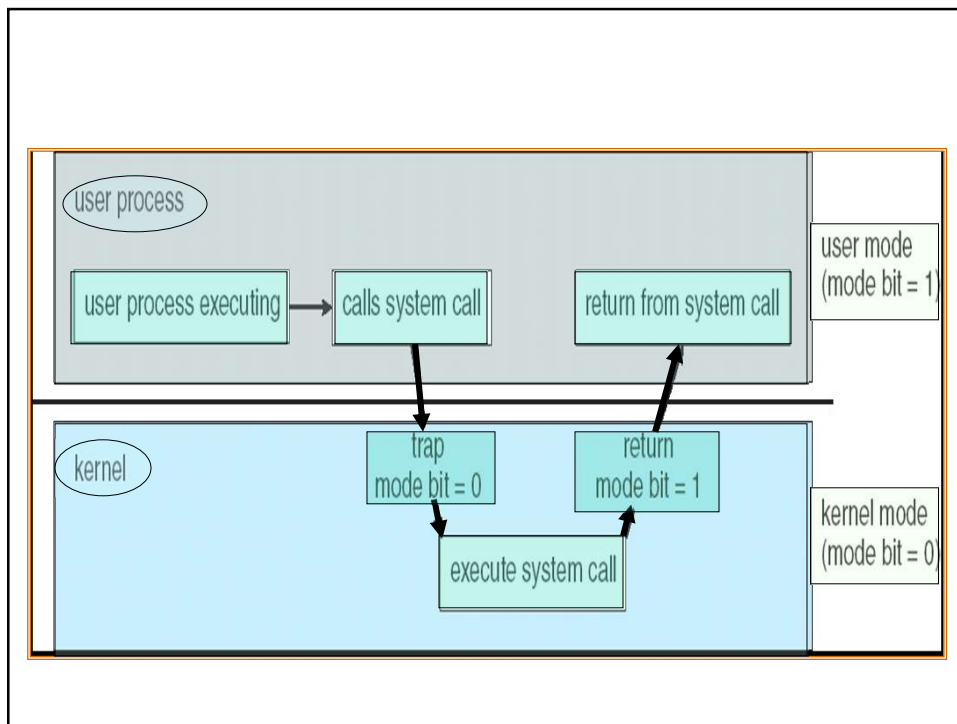
- Interrupt
 - driven by hardware → Handled by OS
- SW/HW error or request
 - creates exception or trap → Handled by OS
 - Division by zero, request for operating system service
 - Device Failure
- Other process problems include
 - infinite loop
 - processes modifying each other or the operating system
- OS acts as controller or supervisor
 - Mechanism → Dual-mode
 - allows OS to protect itself and other system components → User Mode or Kernel Mode

35

Operating System Structures and Operations

- User mode and kernel mode → mode Bit → provided by hardware
 - provides ability to distinguish when system is running user code or kernel code
 - some instructions designated as privileged, only executable in kernel mode
 - system call changes mode to kernel, return from call resets it to user

36



37

Operating System Structures and Operations

- **Timer**
 - prevent infinite loop / process hogging resources
 - Provide by hardware
 - Set interrupt after specific period
 - Operating system decrements counter
 - When counter → zero generate an interrupt
 - Set up before scheduling process to regain control or terminate program that exceeds allotted time

38

Management Activities → Process Management

- **Process**

- a program in execution
- a unit of work within the system
- program is a passive entity
- process is an active entity

- **Process**

- needs resources to accomplish its task
 - CPU, memory, I/O, files
 - Initialization data

- **Process**

- termination requires reclaim of any reusable resources

39

Management Activities → Process Management

- **Single-threaded process**

- has one program counter specifying location of next instruction to execute
- Process executes instructions sequentially, one at a time, until completion

- **Multi-threaded process**

- one program counter per thread

- **Typically system has many**

- processes
- User
- CPUs

40

Management Activities → Process Management

- OS responsibilities
 1. process Creating and deletion
 2. process suspension and resumption
 3. process synchronization
 4. process communication
 5. deadlock handling

41

Management Activities → Memory Management

- All
 - data in memory before and after processing
 - instructions in memory in order to execute
- Optimization
 - Memory management determines what is in memory
 - Maximum CPU utilization
 - Minimize computer response to users
- Memory management activities
 - Keeping track of which parts of memory are currently being used and by whom
 - Deciding which processes (or parts thereof) and data to move into and out of memory
 - Allocating and deallocating memory space as needed

42

Management Activities→ Storage Management

- OS
 - provides uniform, logical view of information storage
- File
 - Abstracts physical properties to logical storage unit
- Medium
 - is controlled by device (i.e., disk drive, tape drive)
 - Varying properties include access speed, capacity, data-transfer rate, access method (sequential or random)
- File-System management
 - Files usually organized into directories
 - Access control on most systems to determine who can access what

43

Management Activities→ Storage Management

- OS activities include
 - Creating and deleting files and directories
 - Primitives to manipulate files and dirs
 - Mapping files onto secondary storage
 - Backup files onto stable (non-volatile) storage media

44

Mass-Storage Management

- Usually, disks used to store data
 - that does not fit in main memory
 - data that must be kept for a “long” period of time.
- Proper management is of central importance
- Entire speed of computer operation
 - hinges on disk subsystem and its algorithms
- OS activities
 - Free-space management
 - Storage allocation
 - Disk scheduling
- Some storage need not be fast
 - Tertiary storage includes optical storage, magnetic tape
 - Still must be managed
 - Varies between WORM (write-once, read-many-times) and RW (read-write)

45

I/O Subsystem

- One purpose of OS is to hide details of hardware devices from the user → IO subsystem
 - I/O subsystem responsible for
 - Memory management of I/O including
 1. buffering → storing data temporarily while it is being transferred
 2. caching → storing parts of data in faster storage for performance
 3. spooling → the overlapping of output of one job with input of other jobs
 - Providing
 - general device-driver interface
 - drivers for specific hardware devices

46

Protection and Security

- **Protection**
 - any mechanism for controlling access of processes or users to resources defined by the OS
 1. Memory; 2. CPU; 3. Storage
- **Security**
 - defense of the system against internal and external attacks
 1. denial-of-service
 2. worms or viruses
 3. identity theft
 4. service theft

47

Protection and Security

- **Security of the Systems**
 - generally, first distinguish among users, to determine who can do what
 - **User identifications (user IDs, security IDs)**
 - include name and associated number, one per user
 - **Mapping**
 - then associated with all files, processes of that user to determine access control
 - **Group identifier (group ID)**
 - allows set of users to be defined and controls managed, then also associated with each process, file
 - **Privilege escalation**
 - Temporary change in user rights

48

Self Study

- Distributed Systems
- Special Purpose Systems
 - Real-time Embedded Systems
 - Multimedia Systems
 - Handheld Systems

49

Computing Environments

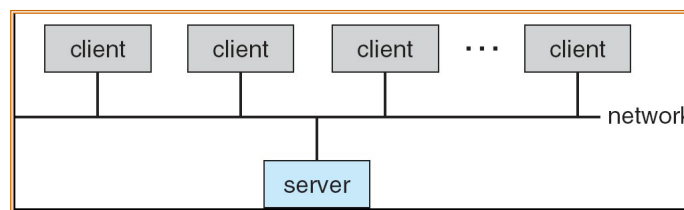
- Traditional computer → Blurring over time
 - Office environment
 - PCs connected to a network; terminals attached to mainframe or minicomputers providing batch and timesharing
 - Now portals allowing networked and remote systems access to same resources
 - Home networks
 - Used to be single system, then modems
 - Now firewalled, networked

50

Computing Environments (Cont.)

□ Client-Server Computing

- Dumb terminals supplanted by smart PCs
- Many systems now **servers**, responding to requests generated by **clients**
 - ▶ **Compute-server** provides an interface to client to request services (i.e. database)
 - ▶ **File-server** provides interface for clients to store and retrieve files

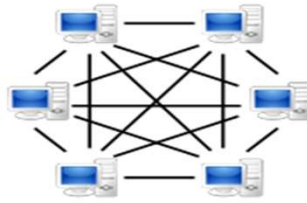


51

Peer-to-Peer Computing

- Another model of distributed system
- P2P does not distinguish clients and servers
 - Instead, all nodes are considered peers
 - May each act as client, server or both
 - Node must join P2P network
 - Register its service with central lookup service on network, or
 - Broadcast request for service and respond to requests for service via discovery protocol

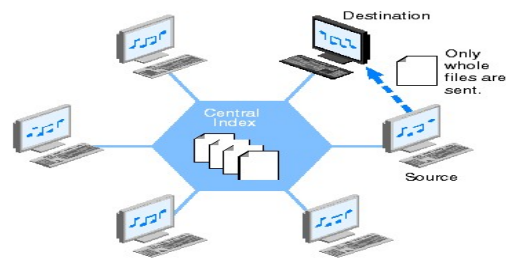
52



From Computer Desktop Encyclopedia
© 2004 The Computer Language Co., Inc.

THE ORIGINAL NAPSTER

Napster provided a central directory of users who had files to share.



53

Web-Based Computing

- Web has become ubiquitous
- PCs most prevalent (common) devices
- More devices becoming networked to allow web access
- New category of devices to manage web traffic among similar servers: **load balancers**
- Use of operating systems like Windows 95, client-side
 - have evolved into Linux and Windows XP, which can be clients and servers

54

Implementation

- Client-Server Computing → TCP/IP
 - Server-Side API's
 - Client-Side API's
- Peer-to-Peer Computing → TCP/IP
 - Server-Side API's
 - Client-Side API's
- Web-Based Computing → HTTP
 - Web server
 - Client server

55

Operating System Services

1. Set of operating-system services
 - provides functions that are helpful to the user
 - User interface
 - Almost all operating systems have a user interface (UI)
 - Command-Line (CLI), Graphics User Interface (GUI), Batch
 - Program execution
 - OS must be able to load a program into memory
 - run that program
 - end execution, either normally or abnormally (indicating error)
 - I/O operations
 - A running program may require I/O, which may involve a file or an I/O device.
 - File-system manipulation
 - The file system is of particular interest. Obviously, programs need to read and write files and directories, create and delete them, search them, list file information, permission management.

56

Operating System Services

1. set of operating-system services
 - provides functions that are helpful to the user
 - **Communications**
 - Processes may exchange information, on the same computer or between computers over a network
 - Communications may be via
 - shared memory
 - message passing (packets moved by the OS)
 - **Error detection** – OS needs to be constantly aware of possible errors
 - May occur in the CPU and memory hardware, in I/O devices, in user program
 - For each type of error, OS should take the appropriate action to ensure correct and consistent computing
 - Debugging facilities can greatly enhance the user's and programmer's abilities to efficiently use the system

57

Operating System Services

2. set of OS functions exists for ensuring
 - the efficient operation of the system itself via resource sharing
 - **Resource allocation** - When multiple users or multiple jobs running concurrently, resources must be allocated to each of them
 - Many types of resources
 - CPU cycles
 - Main memory
 - file storage
 - **Accounting**
 - keep track of which users use how much and what kinds of computer resources
 - **Protection and security**
 - The owners of information stored in a multiuser or networked computer system may want to control use of that information, concurrent processes should not interfere with each other
 - **Protection** involves ensuring that all access to system resources is controlled
 - **Security** of the system from outsiders requires user authentication, extends to defending external I/O devices from invalid access attempts
 - If a system is to be protected and secured, precautions must be instituted throughout it

58

Interaction with OS (OS Interface)

- Command Line Interpreter → CLI
 - direct command entry
 - Sometimes implemented in
 - Kernel → DOS
 - systems program → Windows XP and later version
 - Shells → Unix
 - Primarily designed
 - To fetch user command and execute it
 - » Built-in
 - » Programs

59

Interaction with OS

- Graphical User Interface → GUI
 - User-friendly desktop metaphor interface
 - Usually mouse, keyboard, and monitor
 - Icon represent files, programs, actions, etc.
 - Various mouse buttons over objects in the interface cause various actions
 - information, options, execute function, open directory
- Many systems now include both CLI and GUI interfaces
 - Microsoft Windows is GUI with CLI “command” shell
 - Apple Mac OS X as “Aqua” GUI interface with UNIX kernel underneath and shells available
 - Solaris is CLI with optional GUI interfaces (Java Desktop, KDE)

60

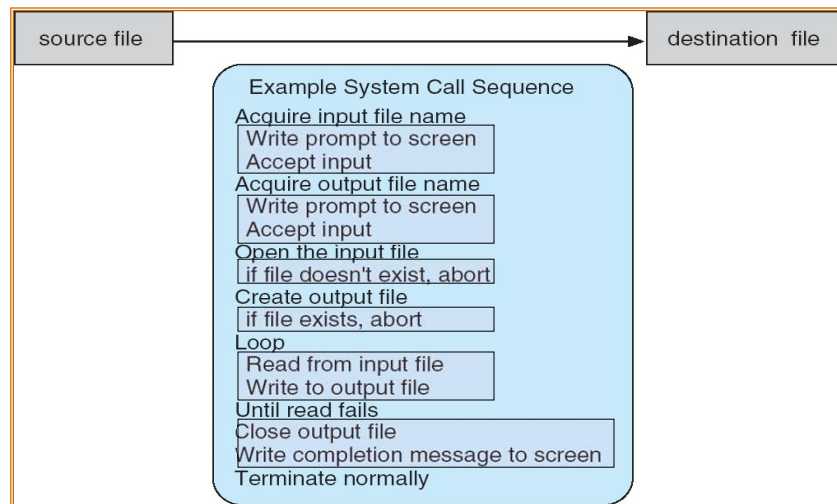
Interaction with OS

- **System Calls**→ ?
 - **Programming interface**
 - services provided by the OS
 - Typically written
 - a high-level language (C ,C++,C#)
 - Mostly accessed
 - by programs via a high-level **Application Program Interface (API)** rather than direct system call use
 - Three most common APIs
 - Win32/64 API for Windows
 - POSIX API for POSIX-based systems (including virtually all versions of UNIX, Linux, and Mac OS X)
 - Java API for the Java virtual machine (JVM)

61

Example: System Calls

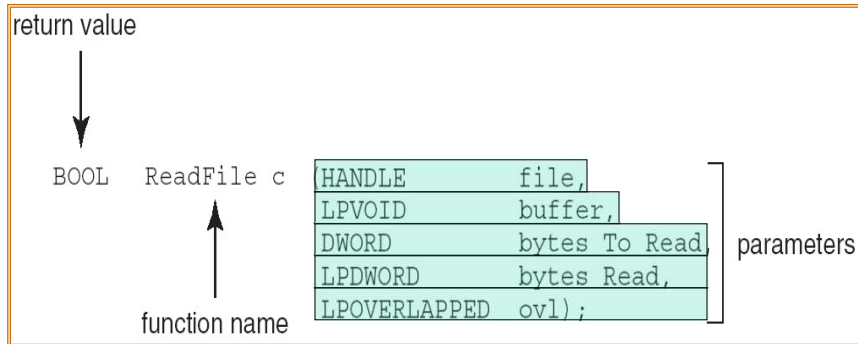
- System call sequence to copy the contents of one file to another file



62

Example of Standard API

- Consider the ReadFile() function in the
 - Win64/32 API—a function for reading from a file

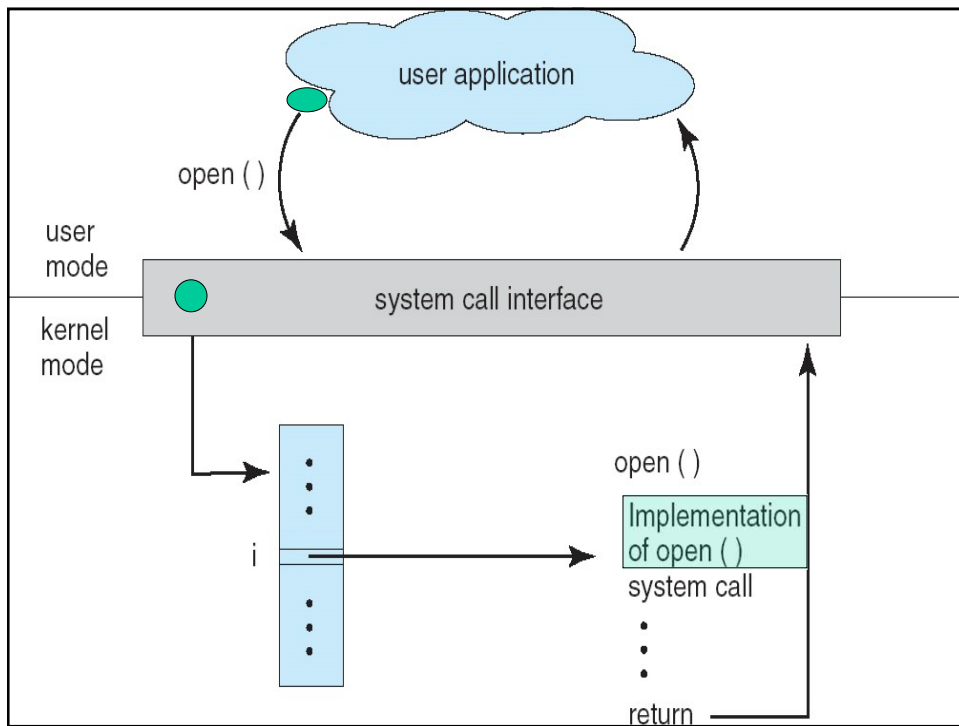


63

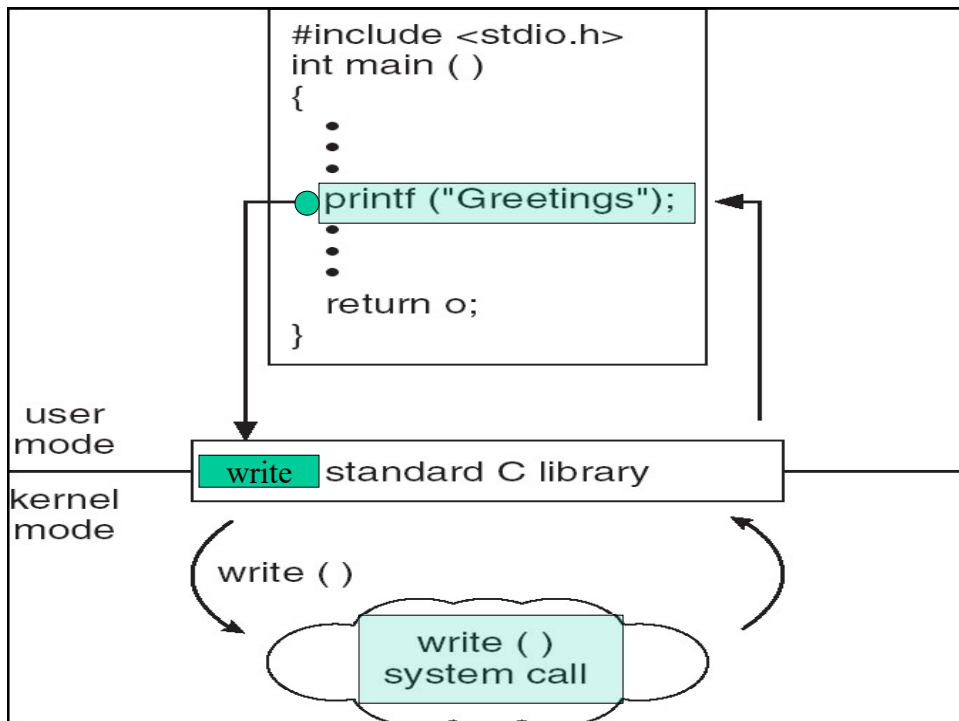
System Call Implementation

- System Call How → ?
 - a number associated with each system call
 - system-call interface maintains a table indexed according to these numbers
- System Call Interface
 - invokes intended system call in OS kernel
 - returns status of the system call and any return values
- caller needs to
 - know nothing about how the system call is implemented
 - obey API and understand
 - Know what OS will do as a result call
- Most details of OS interface hidden from programmer by API
 - Managed by run-time support library (set of functions built into libraries included with compiler)

64



65



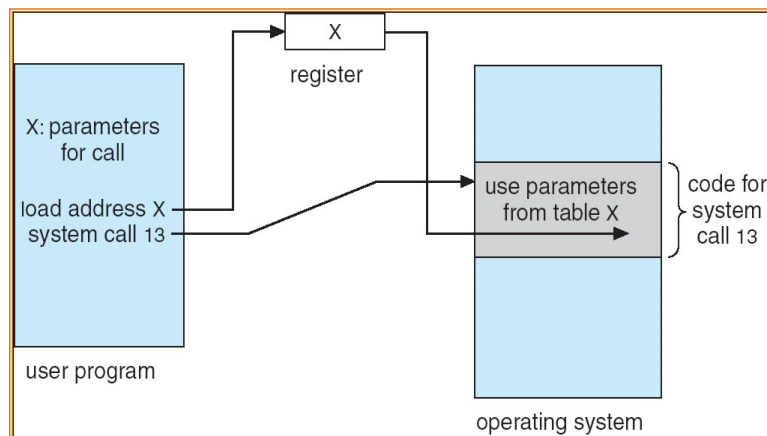
66

System Call → Parameter

- Three methods
 - 1. Registers
 - limit the number or length of parameters being passed
 - 2. Blocks
 - Linux and Solaris
 - do not limit the number or length of parameters being passed
 - 3. Stacks
 - do not limit the number or length of parameters being passed

67

Parameter Passing via Table



68

Types of System Calls

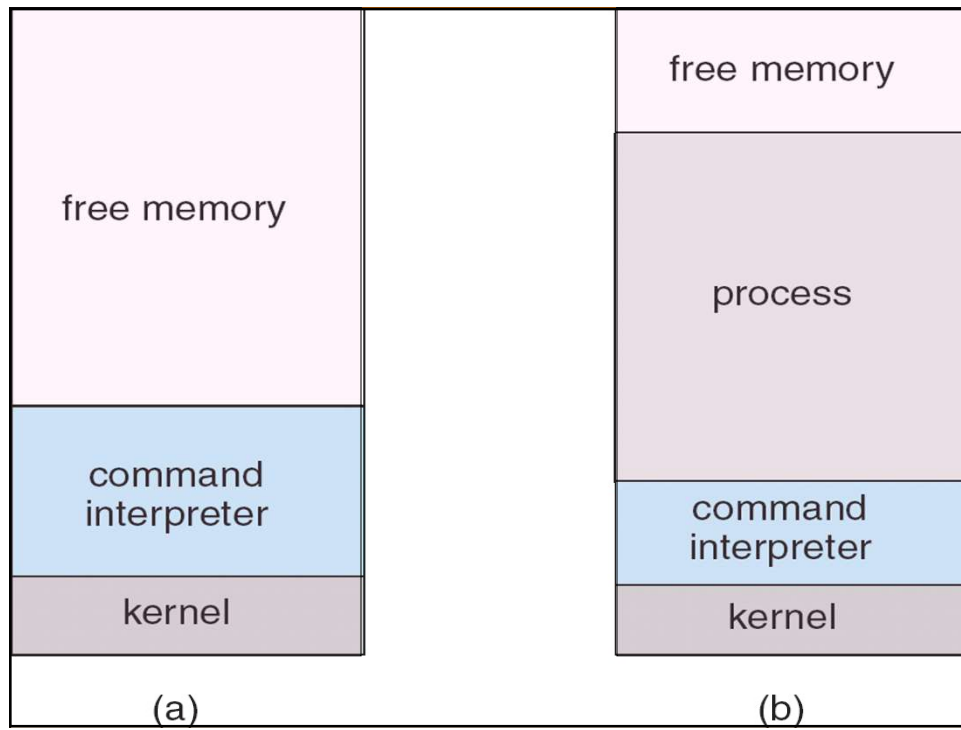
- Process Control
 1. End, abort
 2. Load, Execute
 3. Create Process, Terminate Process
 4. Get Process Attribute; Set Process Attribute
 5. Weight Event; Signal Event
 6. Allocate and free Memory
- File Management
 1. Create File; Delete File
 2. Open; Close
 3. Read; Write; Reposition
 4. Get File Attribute; Set File Attribute

69

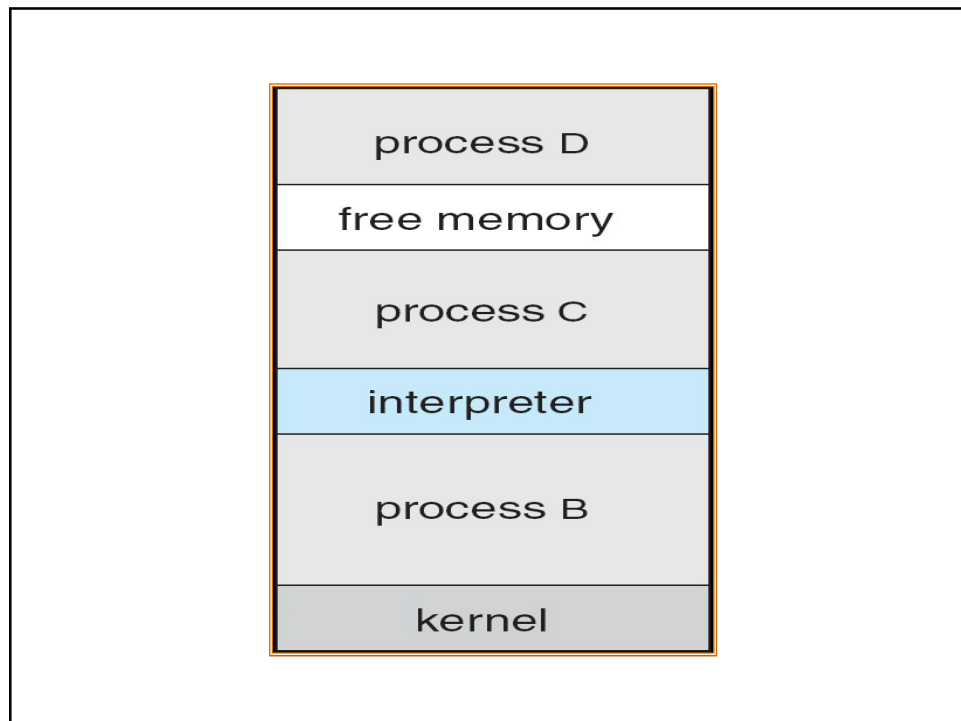
Types of System Calls

- Device Management
 1. Request Device; Release Device
 2. Read; Write; Reposition
 3. Get Device Attribute; Set Device Attribute
 4. Logically Attach or Detach Device
- Information Maintenance
 1. Get/Set Time or Date
 2. Get/Set System Date
 3. Get/Set Process, Device, or File Attributes
- Communication
 1. Create, Delete Communication Connection
 2. Send, Receive Messages
 3. Transfer Status Information
 4. Attach/Detach Remote Devices

70



71



72

System Programs

- OS
 - Collection of system programs
 - Systems Programs → System Utilities
 - Provides convenient Environment
 - Program Development and Execution
- System Programs → Categories
 - File manipulation
 - Status information
 - File modification
 - Programming language support → compiler; assembler; debugger; interpreter
 - Program loading and execution → absolute loader; relocate-able loader; linkage editor; overlay loader
 - Communications
 - Application programs
- Most users' view of the operation system is defined by system programs, not the actual system calls

73

Program Execution and Loading Support

1. a program that transforms source code into another computer language
2. a program that translate assemble code into machine code
3. a computer program that is used to test and debug other programs
4. translates source code into some efficient intermediate representation (code) and immediately executes
5. loads executable with absolute positions of subroutines
6. loads executables with relocate-able positions of subroutines
7. an editor program that creates one module from several by resolving cross-references among the modules
8. an editor that can also be used to build packages of subroutines or other control sections that are generally used together

74

Operating System Design and Implementation

- Design and Implementation
 - OS not “solvable”, but some approaches have proven successful
- Internal Structure
 - Operating Systems can vary widely
- Operating System Design and Implementation
 - Start by
 - defining goals and specifications
 - Affected by
 - choice of hardware, type of system
 - Goals
 - User Goals operating system should be
 - convenient to use, easy to learn, reliable, safe, and fast
 - System goals – operating system should be
 - easy to design, implement, and maintain, as well as flexible, reliable, error-free, and efficient

75

Operating System Design and Implementation

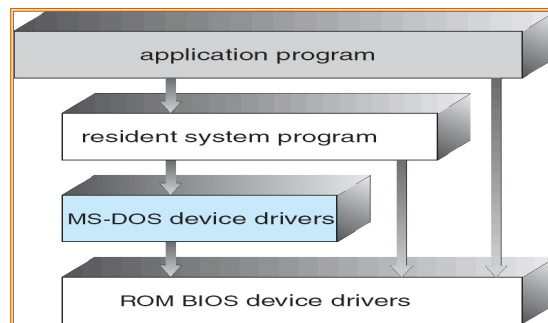
- Important principle to separate
 - Policy: What will be done?
 - Mechanism: How to do it?
 - The separation of policy from mechanism is a very important principle
 - it allows maximum flexibility if policy decisions are to be changed later

76

Operating System Design and Implementation

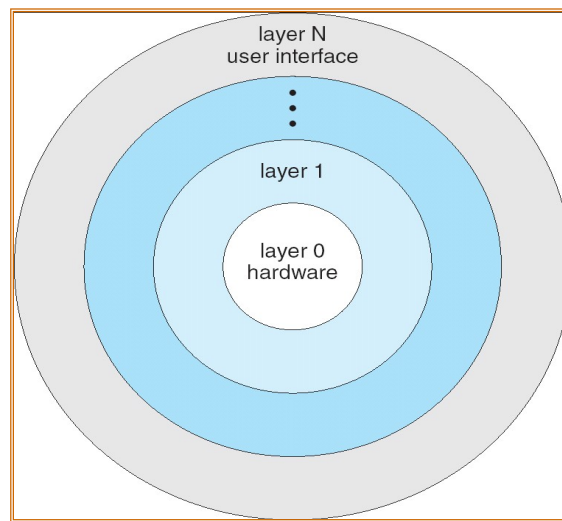
- MS-DOS → Layered Structure

- written to provide the most functionality in the least space
- Not divided into modules
- Although MS-DOS has some structure, its interfaces and levels of functionality are not well separated



77

Layered Operating System



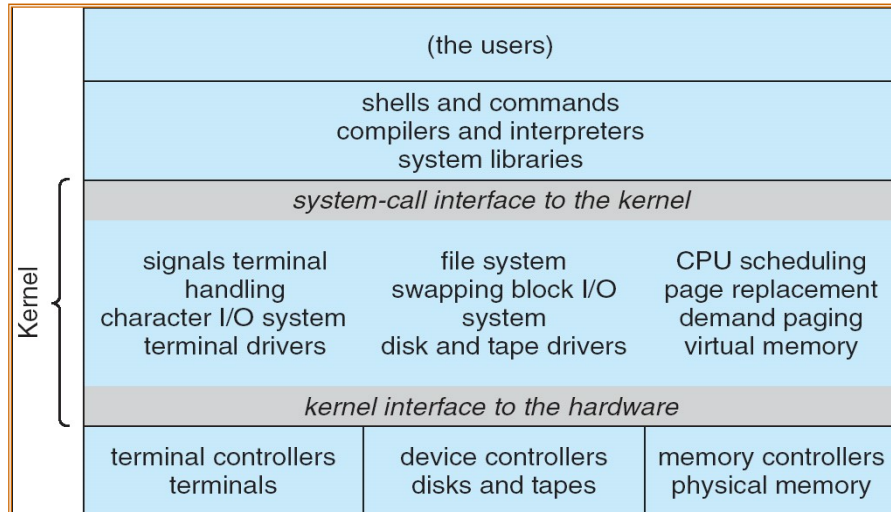
78

UNIX

- limited by hardware functionality
- the original UNIX operating system had limited structuring
- The UNIX OS consists of two separable parts
 - Systems programs
 - The kernel
 - Consists of everything below the system-call interface and above the physical hardware
 - Provides the file system, CPU scheduling, memory management, and other operating-system functions

79

UNIX System Structure



80

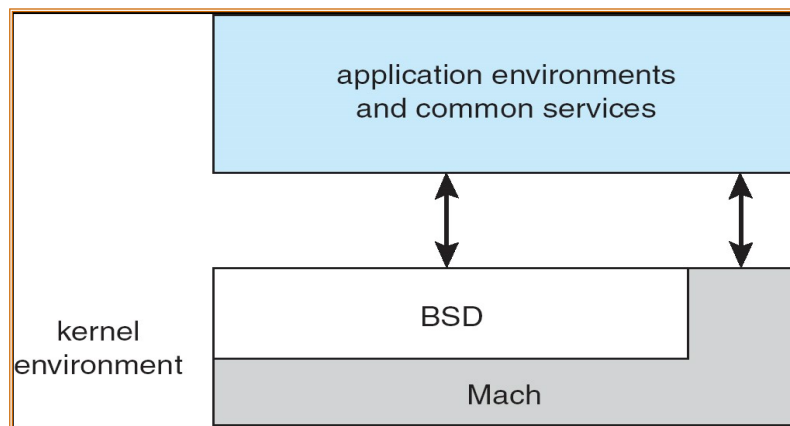
Microkernel System Structure

- Moves portion of kernel into user space
- Communication takes place between user modules using message passing
- Benefits
 - Easier to extend a microkernel
 - Easier to port the operating system to new architectures
 - More reliable (less code is running in kernel mode)
 - More secure
- Drawbacks
 - Performance overhead of user space to kernel space communication

81

Microkernel System Structure

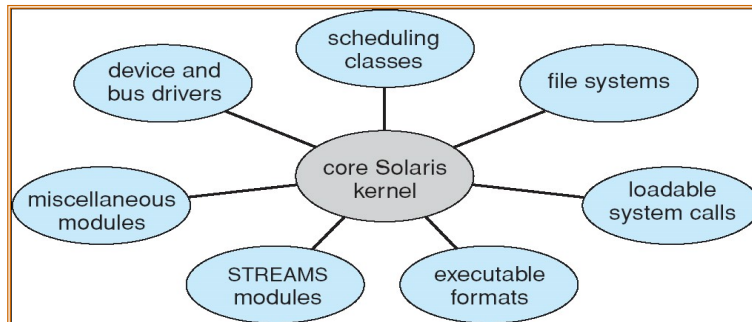
- Example → Mac OS



82

Modular Approach

- Most modern operating systems implement kernel modules
 - Uses object-oriented approach
 - Each core component is separate
 - Each talks to the others over known interfaces
 - Each is loadable as needed within the kernel
- Overall, similar to layers but with more flexible



83

Self Study & Discussion

- Operating System Generation
- SYSGEN program
- Booting
- Bootstrap program
- bootstrap loader
- boot block
- initial boot code

84

```

using System;
using System.Net.Sockets;
using System.Text;
using System.Collections;
namespace server
{
    class Program {
        static void Main(string[] args)
        {
            string myip = "127.0.0.1";
            System.Net.IPAddress localaddress = System.Net.IPAddress.Parse(myip);
            TcpListener serverSocket = new TcpListener(localaddress, 8888);
            TcpClient clientSocket = default(TcpClient);
            serverSocket.Start();
            Console.WriteLine("Chat Server Started ....");
            clientSocket = serverSocket.AcceptTcpClient();
            string dataFromClient = null;
            while (dataFromClient != "EXIT")
            {
                byte[] bytesFrom = new byte[10025];
                NetworkStream networkStream = clientSocket.GetStream();
                networkStream.Read(bytesFrom, 0, (int)clientSocket.ReceiveBufferSize);
                dataFromClient = System.Text.Encoding.ASCII.GetString(bytesFrom);
                dataFromClient = dataFromClient.Substring(0, dataFromClient.IndexOf("$"));
                Console.WriteLine(dataFromClient);
            }
            clientSocket.Close();
            serverSocket.Stop();
            Console.WriteLine("SERVER CLOSED");
            Console.ReadLine();
        }
    }
}

```

85

```

using System;
using System.Net.Sockets;
using System.Text;
using System.Collections;
namespace client1
{
    class Program
    {
        static void Main(string[] args)
        {
            System.Net.Sockets.TcpClient clientSocket = new System.Net.Sockets.TcpClient();
            NetworkStream serverStream = default(NetworkStream);
            clientSocket.Connect("127.0.0.1", 8888);
            serverStream = clientSocket.GetStream();
            string msg = Console.ReadLine();
            while (msg != null)
            {
                byte[] outStream = System.Text.Encoding.ASCII.GetBytes(msg.ToString() + "$");
                serverStream.Write(outStream, 0, outStream.Length);
                serverStream.Flush();
                msg = Console.ReadLine();
            }
            clientSocket.Close();
            serverStream.Close();
        }
    }
}

```

86